that will help you progressively discover and deliver an effective solution that matches up with the underlying business goals behind a project. As you'll see, one important benefit of BDD is that it provides techniques that can help you manage this uncertainty and reduce the risk that comes with it.

1.3 Introducing Behavior-Driven Development

Behavior-Driven Development (BDD) is a set of software engineering practices designed to help teams build and deliver more valuable, higher quality software faster. It draws on Agile and lean practices including, in particular, Test-Driven Development (TDD) and Domain-Driven Design (DDD). But most importantly, BDD provides a common language based on simple, structured sentences expressed in English (or in the native language of the stakeholders) that facilitate communication between project team members and business stakeholders.

To better understand the motivations and philosophy that drive BDD practices, it's useful to understand where BDD comes from.

1.3.1 BDD was originally designed as an improved version of TDD

BDD was originally invented by Dan North[9] in the early to mid-2000s as an easier way to teach and practice Test-Driven Development (TDD). TDD, invented by Kent Beck in the early days of Agile,[10] is a remarkably effective technique that uses unit tests to specify, design, and verify application code.

When TDD practitioners need to implement a feature, they first write a failing test that describes, or specifies, that feature. Next, they write just enough code to make the test pass. Finally, they refactor the code to help ensure that it will be easy to maintain (see figure 1.5). This simple but powerful technique encourages developers to write cleaner, better-designed, easier-to-maintain code[11] and results in substantially lower defect counts.[12]

Despite its advantages, many teams still have difficulty adopting and using TDD effectively. Developers often have trouble knowing where to start or what tests they should write next. Sometimes TDD can lead developers to become too detail-focused, losing the broader picture of the business goals they're supposed to implement. Some teams also find that the large numbers of unit tests can become hard to maintain as the project grows in size.

In fact, many traditional unit tests, written with or without TDD, are tightly coupled to a particular implementation of the code. They focus on the method or function they're testing, rather than on what the code should do in business terms.

---

[9] Dan North, "Introducing BDD," http://dannorth.net/introducing-bdd/.

[10] Kent Beck, *Test-Driven Development: By Example* (Addison-Wesley Professional, 2002).

[11] Rod Hilton, "Quantitatively Evaluating Test-Driven Development by Applying Object-Oriented Quality Metrics to Open Source Projects" (PhD thesis, Regis University, 2009), http://www.rodhilton.com/files/tdd_thesis.pdf.

[12] Nachiappan Nagappan et al., "Realizing Quality Improvement through Test Driven Development" (2008), http://research.microsoft.com/en-us/groups/ese/nagappan_tdd.pdf.